Hybrid MPI+OpenMP SOR Solver for Coaxial Potentials

1 Coaxial Conductor

1.1 Discretisation of the Laplace Equation for $r > 0$

For the case that $r > 0$, we discretise the Laplace Equation in cylindrical polar coordinates given by

$$\nabla^2\Phi = \frac{\partial^2\Phi}{\partial z^2} + \frac{1}{r}\frac{\partial\Phi}{\partial r} + \frac{\partial^2\Phi}{\partial r^2} + \frac{1}{r^2}\frac{\partial^2\Phi}{\partial\theta^2} \quad r > 0 \tag{1}$$

Due to rotational symmetry any term involving θ is zero, therefore, we have that

$$\frac{1}{r^2}\frac{\partial^2\Phi}{\partial\theta^2} = 0 \tag{2}$$

Therefore, the Laplace Equation becomes

$$\frac{\partial^2\Phi}{\partial z^2} + \frac{1}{r}\frac{\partial\Phi}{\partial r} + \frac{\partial^2\Phi}{\partial r^2} = 0 \tag{3}$$

Applying the centred-difference approximation to each derivative gives the following equations

$$\frac{\partial^2\Phi}{\partial z^2} \approx \frac{\Phi(z+h) - 2\Phi(z) + \Phi(z-h)}{h^2} = \frac{\Phi_{i+1,k} - 2\Phi_{i,k} + \Phi_{i-1,k}}{h^2} \tag{4}$$

$$\frac{\partial^2\Phi}{\partial r^2} \approx \frac{\Phi(r+h) - 2\Phi(r) + \Phi(r-h)}{h^2} \approx \frac{\Phi_{i,k+1} - 2\Phi_{i,k} + \Phi_{i,k-1}}{h^2} \tag{5}$$

$$\frac{\partial\Phi}{\partial r} \approx \frac{\Phi(r+h) - \Phi(r-h)}{2h} \approx \frac{\Phi_{i,k+1} - \Phi_{i,k-1}}{2h} \tag{6}$$

Then, using the substitution $r = kh$

$$\frac{1}{r}\frac{\partial\Phi}{\partial r} \approx \frac{1}{kh}\left(\frac{\Phi_{i,k+1} - \Phi_{i,k-1}}{2h}\right) \approx \frac{\Phi_{i,k+1} - \Phi_{i,k-1}}{2kh^2} \tag{7}$$

Substituting the derivatives back into the Laplace equation gives

$$\frac{\partial^2\Phi}{\partial z^2} + \frac{1}{r}\frac{\partial\Phi}{\partial r} + \frac{\partial^2\Phi}{\partial r^2} \approx \frac{\Phi_{i+1,k} - 2\Phi_{i,k} + \Phi_{i-1,k}}{h^2} + \frac{\Phi_{i,k+1} - 2\Phi_{i,k} + \Phi_{i,k-1}}{h^2} + \frac{\Phi_{i,k+1} - \Phi_{i,k-1}}{2kh^2} = 0 \tag{8}$$

Multiplying through by h^2 removes the denominator

$$\Phi_{i+1,k} - 2\Phi_{i,k} + \Phi_{i-1,k} + \Phi_{i,k+1} - 2\Phi_{i,k} + \Phi_{i,k-1} + \frac{1}{2k}(\Phi_{i,k+1} - \Phi_{i,k-1}) \tag{9}$$

Collecting like terms gives

$$-4\Phi_{i,k} + (\Phi_{i+1,k} + \Phi_{i-1,k} + \Phi_{i,k+1} + \Phi_{i,k-1}) + \frac{1}{2k}(\Phi_{i,k+1} - \Phi_{i,k-1}) = 0 \tag{10}$$

Then, by moving the $-4\Phi_{i,k}$ term to the other side and dividing by 4, we arrive at the solution

$$\Phi_{i,k} = \frac{1}{4}(\Phi_{i+1,k} + \Phi_{i-1,k} + \Phi_{i,k+1} + \Phi_{i,k-1}) + \frac{1}{8k}(\Phi_{i,k+1} - \Phi_{i,k-1}) \tag{11}$$

1.2 Discretisation of the Laplace Equation for $r = 0$

We aim to discretise the Laplace Equation for $r = 0$. The singularity at $r = 0$ can be avoided by the use of Cartesian coordinates.

The Laplace Equation in Cartesian coordinates is given by

$$\nabla^2\Phi = \frac{\partial^2\Phi}{\partial z^2} + \frac{\partial^2\Phi}{\partial x^2} + \frac{\partial^2\Phi}{\partial y^2} = 0 \tag{12}$$

Considering rotational symmetry about the z-axis gives the following relation

$$\frac{\partial^2\Phi}{\partial x^2} = \frac{\partial^2\Phi}{\partial y^2} = \frac{\partial^2\Phi}{\partial r^2} \tag{13}$$

Therefore, the Laplace Equation becomes

$$\nabla^2\Phi = \frac{\partial^2\Phi}{\partial z^2} + 2\frac{\partial^2\Phi}{\partial r^2} = 0 \tag{14}$$

Again, applying the centred-difference approximation to the derivatives and substituting these into the Laplace Equation gives

$$\frac{\Phi_{i+1,0} - 2\Phi_{i,0} + \Phi_{i-1,0}}{h^2} + 2\left(\frac{\Phi_{i,1} - 2\Phi_{i,0} + \Phi_{i,-1}}{h^2}\right) = 0 \tag{15}$$

Considering that $\Phi_{i,-1} = \Phi_{i,1}$ must be true due to symmetry, and multiplying through by h^2 to remove the denominator gives

$$\Phi_{i+1,0} - 2\Phi_{i,0} + \Phi_{i-1,0} + 2(2\Phi_{i,1} - 2\Phi_{i,0}) = 0 \tag{16}$$

Collecting like terms and rearranging gives

$$6\Phi_{i,0} = 4\Phi_{i,1} + \Phi_{i+1,0} + \Phi_{i-1,0} \tag{17}$$

Finally, dividing through by 6, we arrive at the solution

$$\Phi_{i,0} = \frac{2}{3}\Phi_{i,1} + \frac{1}{6}(\Phi_{i+1,0} + \Phi_{i-1,0}) \tag{18}$$

1.3 Hybrid MPI+OpenMP Implementation

A hybrid MPI + OpenMP code (Listing 7) was written to solve the electrostatic potential problem to millivolt accuracy using the successive over-relaxation (SOR) method. The solver combines distributed memory parallelism (MPI) for domain decomposition across nodes and shared memory parallelism (OpenMP) for loop-level acceleration within nodes.

The simulation domain ($z \in [0, L], r \in [0, R]$) is decomposed using a 1d column-wise decomposition along the radial axis (r). The global grid size of $N_z \times N_r$ is split into P vertical strips, where each MPI rank owns a sub-grid of size $N_z \times (N_r/P)$.

Each rank allocates two extra columns (at indices 0 and $N_{local} + 1$) to store halo data from neighbouring ranks. These are updated once per iteration phase using non-blocking MPI communications (`MPI_Isend` and `MPI_Irecv`). To prevent race conditions during the parallel update, the grid points are divided into 'red' and 'black' sets in a checkerboard pattern. This decouples dependencies, allowing all 'red' points to be updated simultaneously (using OpenMP threads) while 'black' points remain fixed, and vice versa. To mitigate the latency cost of MPI, the solver attempts to hide communication behind computation. The algorithm initiates non-blocking halo exchanges first, then immediately proceeds to update the inner bulk of the local domain (which

requires no external data). Only after the inner bulk is finished does the code `MPI_Waitall` and update the halo boundary regions.

To ensure the hybrid parallel code produces physically correct results, it was validated against the serial (single-thread) implementation. The code was tested with varying configurations of MPI ranks and OpenMP threads. In all cases, the solver converged in exactly the same number of iterations, confirming that the red-black ordering and halo logic are free of race conditions.

1.3.1 High-Resolution Simulation Results

The code was executed using the high-resolution parameters detailed in Table 1. The resulting potential distribution is visualised in Figure 1, and specific values at the probe locations are recorded in Table 2.

Table 1: Parameters used to generate results and potential plot for the electrostatic simulation.

Parameter	**Symbol**	**Value**	**Description**
Geometry			
Outer Length	L_{outer}	20.0	Length of the outer cylinder
Inner Length	L_{inner}	5.0	Length of the inner conductor
Outer Radius	R_{outer}	10.0	Radius of the outer cylinder
Inner Radius	R_{inner}	1.0	Radius of the inner conductor
Inner Potential	ϕ_{inner}	1000.0	Potential applied to inner conductor (V)
Solver Settings			
Grid Spacing	h	0.002	Discretisation step size
Relaxation Factor	ω	1.98	SOR relaxation parameter
Tolerance	ϵ	1.0×10^{-4}	Convergence tolerance
Max Iterations	N_{max}	1,000,000	Maximum iteration limit

Figure 1 shows a clear decay of electric potential from the inner conductor ($\phi = 1000$ V) towards the grounded outer walls ($\phi = 0$ V).

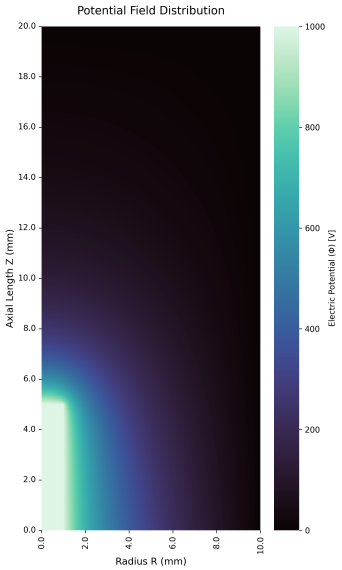


Figure 1: Potential Field distribution of the coaxial cavity.

In the inner conductor region ($z \leq 5.0$ mm $r \leq 1.0$ mm) the plot shows a uniform high-potential. The sharp gradient surrounding this region indicates a strong electric field, which is expected near the conductor's surface where charge density is highest.

The potential smoothly approaches zero at the outer radius ($r = 10.0$ mm) and the top/bottom faces ($z = 0$ and $z = 20.0$ mm), satisfying the boundary conditions .

Point A ($z = 7.5$ mm, $r = 0.0$ mm) located on the central axis just above the tip of the inner conductor ($z = 5.0$), has a potential of 301.6148 V. Although there is no conductor at this location, the proximity to the 1000 V tip maintains a high potential, which decays rapidly as z increases towards the grounded top wall.

Point B ($z = 5.0$ mm, $r = 6.0$ mm) is located at the same axial height as the tip of the inner conductor but at a larger radius. The potential here drops to 148.0249 V, demonstrating the radial $1/r$ decay of the electrical field. Even though this point is horizontally aligned with the high-voltage tip (1000 V), the distance from the axis causes a significant drop in potential compared to the surface of the conductor.

Point C ($z = 12.5$ mm, $r = 5.0$ mm) is located deep in the cavity bulk, far from both the inner conductor and the outer walls. The potential is significantly lower at 39.4380 V. This confirms that the electric field is highly concentrated near the inner conductor and that the potential influence becomes weak in the empty upper volume of the cavity.

Probe Point	**Location (z, r) [mm]**	**Potential [V]**
Point A	$(7.5, 0.0)$	301.6148
Point B	$(5.0, 6.0)$	148.0249
Point C	$(12.5, 5.0)$	39.4380

Table 2: Simulated electric potential at specified probe points ($h = 0.002$, $\omega = 1.98$). The simulation converged with a final error of 9.999×10^{-5}.

1.4 Performance Profiling and Optimisation

The code (Listing 7) was profiled with gprof and compiled with `mpif90 -fopenmp -pg` for both `-O0` and `-O3` optimisation flags. The code was executed on a single core (1 rank, 1 thread) of Viking with the simulation parameters given in Table 3. To ensure a rapid profiling cycle, a coarser resolution ($h = 0.02$) was used compared to the high-resolution production runs.

Table 3: Parameters used to profile the electrostatic simulation.

Parameter	**Symbol**	**Value**	**Description**
Geometry			
Outer Length	L_{outer}	20.0	Length of the outer cylinder
Inner Length	L_{inner}	5.0	Length of the inner conductor
Outer Radius	R_{outer}	10.0	Radius of the outer cylinder
Inner Radius	R_{inner}	1.0	Radius of the inner conductor
Inner Potential	ϕ_{inner}	1000.0	Potential applied to inner conductor (V)
Solver Settings			
Grid Spacing	h	0.02	Discretisation step size
Relaxation Factor	ω	1.9	SOR relaxation parameter
Tolerance	ϵ	1.0×10^{-4}	Convergence tolerance
Max Iterations	N_{max}	200,000	Maximum iteration limit

To establish a baseline, the code was first compiled without optimisation (`-O0`). This prevents the compiler from altering the function hierarchy.

Listing 1: Flat profile output for unoptimised (-O0) execution showing the dominance of main and update_bulk routines.

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 60.76    122.79   122.79                             main
 39.28    202.19    79.39 625130369     0.00     0.00  __potentials_MOD_update_bulk
  0.04    202.26     0.08 14201789     0.00     0.00  __potentials_MOD_update_axis
  0.01    202.28     0.02        1    20.02    20.02  __sim_params_MOD_default_params
  0.00    202.29     0.01        1    10.01    10.01  __sor_solver_MOD_solve
  0.00    202.30     0.01        1     5.01     5.01  __grid_data_MOD_save_parallel_dat
  0.00    202.30     0.00      450     0.00     0.00  __potentials_MOD_coax_analytic
  0.00    202.30     0.00        3     0.00     0.00  __grid_data_MOD_probe_point
  0.00    202.30     0.00        1     0.00    35.04  MAIN__
  0.00    202.30     0.00        1     0.00     0.00  __grid_data_MOD_cleanup
  0.00    202.30     0.00        1     0.00     0.00  __grid_data_MOD_init_grid
```

Listing 2: Call graph for the unoptimised (-O0) execution, showing the hierarchy of function calls originating from main.

```
index % time    self  children    called     name
                                                 <spontaneous>
[1]    100.0  122.79   79.50                 main [1]
                79.39    0.00 625130369/625130369     __potentials_MOD_update_bulk [2]
                 0.08    0.00 14201789/14201789     __potentials_MOD_update_axis [3]
                 0.00    0.04       1/1           MAIN__ [4]
-----------------------------------------------
                79.39    0.00 625130369/625130369     main [1]
[2]     39.2   79.39    0.00 625130369         __potentials_MOD_update_bulk [2]
-----------------------------------------------
                 0.08    0.00 14201789/14201789     main [1]
[3]      0.0    0.08    0.00 14201789         __potentials_MOD_update_axis [3]
-----------------------------------------------
                 0.00    0.04       1/1           main [1]
[4]      0.0    0.00    0.04       1         MAIN__ [4]
                 0.02    0.00       1/1           __sim_params_MOD_default_params [5]
                 0.01    0.00       1/1           __sor_solver_MOD_solve [6]
                 0.01    0.00       1/1           __grid_data_MOD_save_parallel_dat [7]
                 0.00    0.00       3/3           __grid_data_MOD_probe_point [15]
                 0.00    0.00       1/1           __grid_data_MOD_init_grid [17]
                 0.00    0.00       1/1           __grid_data_MOD_cleanup [16]
-----------------------------------------------
                 0.02    0.00       1/1           MAIN__ [4]
[5]      0.0    0.02    0.00       1         __sim_params_MOD_default_params [5]
-----------------------------------------------
                 0.01    0.00       1/1           MAIN__ [4]
[6]      0.0    0.01    0.00       1         __sor_solver_MOD_solve [6]
-----------------------------------------------
                 0.01    0.00       1/1           MAIN__ [4]
[7]      0.0    0.01    0.00       1         __grid_data_MOD_save_parallel_dat [7]
-----------------------------------------------
                 0.00    0.00     450/450         __grid_data_MOD_init_grid [17]
[14]     0.0    0.00    0.00     450         __potentials_MOD_coax_analytic [14]
-----------------------------------------------
                 0.00    0.00       3/3           MAIN__ [4]
[15]     0.0    0.00    0.00       3         __grid_data_MOD_probe_point [15]
-----------------------------------------------
                 0.00    0.00       1/1           MAIN__ [4]
[16]     0.0    0.00    0.00       1         __grid_data_MOD_cleanup [16]
-----------------------------------------------
                 0.00    0.00       1/1           MAIN__ [4]
[17]     0.0    0.00    0.00       1         __grid_data_MOD_init_grid [17]
                 0.00    0.00     450/450         __potentials_MOD_coax_analytic [14]
-----------------------------------------------
```

The unoptimised profile (Listing 1) confirms that the runtime is strictly dominated by the SOR algorithm. The `main` routine (containing the iteration loop overhead) accounts for 60.8% of the time, while the `update_bulk` calculation accounts for 39.3%. Combined, over 99.9% of the execution time is spent in the solver logic, with approximately 6.25×10^8 calls to the update kernel.

The profile also displays the efficiency of the I/O strategy. The `save_parallel`, which writes formatted text files (`.dat`), consumes only 0.01 seconds (0.00% of runtime). This demonstrates that despite using formatted ASCII output, the I/O overhead is negligible compared to the computational cost of the solver.

Enabling maximum optimisation (`-O3`) resulted in a dramatic performance improvement, reducing the total execution time from 202.30 s to 19.00 s (a 10.6× speedup).

Listing 3: Flat profile output for optimised (-O3) execution.

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  Ts/call  Ts/call  name
100.12     19.00    19.00                             __sim_params_MOD___copy_sim_params_Params_t
  0.00     19.00     0.00        3     0.00     0.00  __grid_data_MOD_probe_point
  0.00     19.00     0.00        1     0.00     0.00  __grid_data_MOD_init_grid
  0.00     19.00     0.00        1     0.00     0.00  __grid_data_MOD_save_parallel_dat
  0.00     19.00     0.00        1     0.00     0.00  __sor_solver_MOD_solve
```

Listing 4: Call graph for the optimised (-O3) exectuion.

```
index % time    self  children    called     name
                                                 <spontaneous>
[1]    100.0   19.00    0.00                 __sim_params_MOD___copy_sim_params_Params_t [1]
-----------------------------------------------
                0.00    0.00       3/3           MAIN__ [2]
[10]     0.0    0.00    0.00       3         __grid_data_MOD_probe_point [10]
-----------------------------------------------
                0.00    0.00       1/1           MAIN__ [2]
[11]     0.0    0.00    0.00       1         __grid_data_MOD_init_grid [11]
-----------------------------------------------
                0.00    0.00       1/1           MAIN__ [2]
[12]     0.0    0.00    0.00       1         __grid_data_MOD_save_parallel_dat [12]
-----------------------------------------------
                0.00    0.00       1/1           MAIN__ [2]
[13]     0.0    0.00    0.00       1         __sor_solver_MOD_solve [13]
-----------------------------------------------
```

The optimised profile shows the effect of the `-O3` optimisation by the compiler. As seen in listing 4, the `update_bulk` and `update_axis` functions have disappeared from the call graph. This confirms that the compiler successfully inlined these pure functions directly into the main solver loop. This validates the modular design choice, keeping the physics logic in small, readable functions did not incur function-call overhead in the production binary.

Note: The attribution of 100.12% runtime to `__copy_sim_params` in Listing 3 is a known profiling artifact. Due to aggressive loop unrolling and inlining, `gprof` misattributes the main loop's instruction addresses to the nearest valid symbol table entry.

Guided by the profiling data, the parallelisation effort focused strictly on the identified computational hotspots. The unoptimised profile (Listing 1) identified the `update_bulk` routine and its associated loop overhead as consuming over 99.9% of the execution time. Consequently, this routine was the primary target for parallelisation. Since the boundary updates (`update_axis` and halo regions) constitute less than 0.1% of the runtime, parallelising them provides minimal speedup but was done to maintain load balance and code uniformity.

Standard SOR updates create a data dependency where $\phi_{i,j}$ depends on the newly calculated values of its neighbours (e.g., $\phi_{i-1,j}$). This prevents simple parallelisation. To resolve this, a red-black (checkerboard) ordering scheme was implemented. By splitting the grid into 'red' ($i + j$ is even) and 'black' ($i + j$ is odd) points, the updates for all 'red' points become independent of each other (depending only on fixed 'black' points). This independence allows the inner loops to be safely parallelised with OpenMP threads (`!$OMP PARALLEL DO`) without race conditions, and enables the domain to be split across MPI ranks without needing communication within a half-step.

The grid is decomposed column-wise along the radial axis. This was chosen because the profile shows the workload is uniform across the grid, splitting by columns ensures each rank gets an equal slice of the computational hotspot (the bulk update).

Within each MPI rank, OpenMP threads are assigned to chunks of the inner loops. This reduces the number of MPI messages required (compared to a pure-MPI approach) while saturating the memory bandwidth of the multi-core node.

Beyond compiler flags, several manual optimisations were implemented in the hybrid code to maximise efficiency. To mitigate MPI latency, the solver uses non-blocking communications (`MPI_Isend`/`MPI_Irecv`). The execution flow initiates halo exchanges first, then updates the inner bulk of the local domain (which has no dependencies), and only waits for MPI messages (`MPI_Waitall`) when ready to update the boundary columns. This keeps the CPU active while data traverses the network.

To eliminate-file system contention a file-per-rank output routine (`save_parallel_dat`) was developed. Each MPI rank writes its local data partition to a unique file (e.g., `phi_rank_0.dat`, `phi_rank_1.dat`) simultaneously. These files can then be combined back together using post-processing scripts.

As Fortran stores arrays in column-major order, the loops were nested such that the z-index (rows) is the inner loop and the r-index (columns) is the outer loop. This ensures stride-1 memory access, minimising cache misses and facilitating vectorisation.

The expensive floating-point division required for the radial factor $1/(8r)$ is pre-calculated as

`inv_r_factor` at the start of each column loop, replacing a division operation inside the innermost loop with a significantly faster multiplication.

1.5 Scaling Analysis and Performance Evaluation

All benchmarks were conducted on the Viking cluster. The simulation code was compiled using `mpif90 -O3 -fopenmp` with OpenMP flags `export OMP_NUM_THREADS=$t`, `export OMP_PLACES=cores` and `export OMP_PROC_BIND=close` to ensure reproducible thread placement. Additionally, the MPI flag `-map-by socket:PE=$t` was utilised to manage socket-level distribution. The execution script is provided in Listing 5.

The scaling runs utilised the parameters defined in Table 4 and the grid resolutions outlined in Table 5. The relaxation parameter was held constant at $\omega = 1.98$.

Table 4: Parameters used for MPI+OpenMP scaling run on Viking.

Parameter	**Symbol**	**Value**	**Description**
Geometry			
Outer Length	L_{outer}	20.0	Length of the outer cylinder
Inner Length	L_{inner}	5.0	Length of the inner conductor
Outer Radius	R_{outer}	10.0	Radius of the outer cylinder
Inner Radius	R_{inner}	1.0	Radius of the inner conductor
Inner Potential	ϕ_{inner}	1000.0	Potential applied to inner conductor (V)
Solver Settings			
Grid Spacing	h	0.02, 0.005, 0.002	Discretisation step sizes
Relaxation Factor	ω	1.98	SOR relaxation parameter
Tolerance	ϵ	1.0×10^{-4}	Convergence tolerance
Max Iterations	N_{max}	1,000,000	Maximum iteration limit

Table 5: Grid resolution parameters used for the scaling analysis.

Case Name	**Grid Dimensions** $(N_z \times N_r)$	**Grid Spacing** (h)	**Total Points**
Coarse	1000×500	0.02	5×10^5
Medium	4000×2000	0.005	8×10^6
Ultrafine	10000×5000	0.002	5×10^7

1.5.1 Strong Scaling

Figure 2a presents the strong scaling metrics for the three grid resolutions. At low processor counts (up to 2 processing elements), all configurations exhibit near-ideal scaling, indicating that the workload per process is sufficiently large to mask initial parallel overhead and that the domain decomposition is correctly implemented.

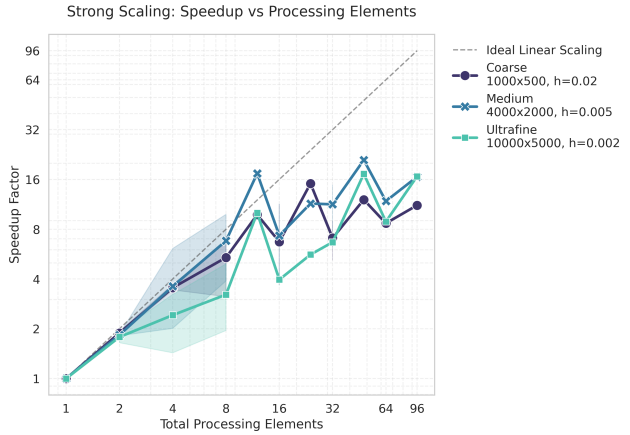
As core counts increase, the coarse grid saturates rapidly beyond 16 cores (Figure 2a). In this regime, the subdomain size becomes too small to offset the cumulative costs of MPI halo exchanges and global synchronisations, limiting scalability.

The medium grid demonstrates superior scaling up to 12 processing elements, where it exhibits a dramatic spike to super-linear scaling (speedup $\sim 20\times$). This behaviour is attributed to cache efficiency, where sub-grid chunks fit optimally into the L3 cache, allowing memory access speeds to exceed the serial DRAM baseline. Coincidentally, this super-linear spike corresponds to 12 MPI ranks $\times$ 1 thread, suggesting strong scaling gains with pure MPI.

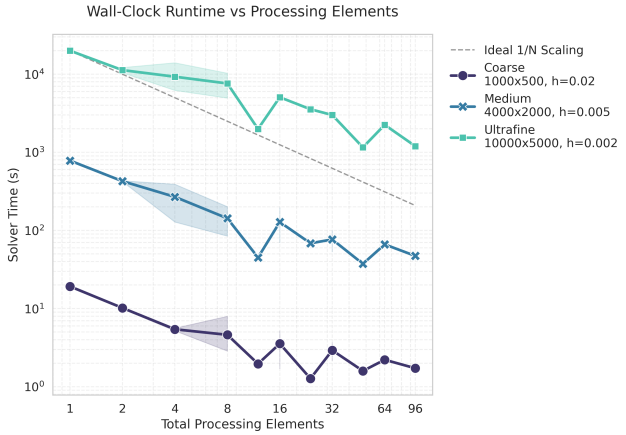
The ultrafine grid initially shows the lowest relative speedup due to massive memory bandwidth pressure, However it sustains positive scaling up to 96 cores, as the higher arithmetic intensity per subdomain allows it to effectively utilise high core counts.

Notably, all configurations deviate from ideal linear scaling at sufficiently large core counts. The solver requires halo exchanges between neighbouring MPI ranks at every iteration, as the number of ranks increases, the surface-to-volume ratio of each subdomain grows, increasing the relative cost of communication. Additionally, the red-black SOR scheme requires synchronisation between phases, and a global reduction is performed each iteration to evaluate convergence. These collective operations impose latency that becomes increasingly significant at scale. Finally, at high thread counts, especially for the ultrafine grid, memory bandwidth saturation and non-uniform memory access (NUMA) effects limit further speedup.

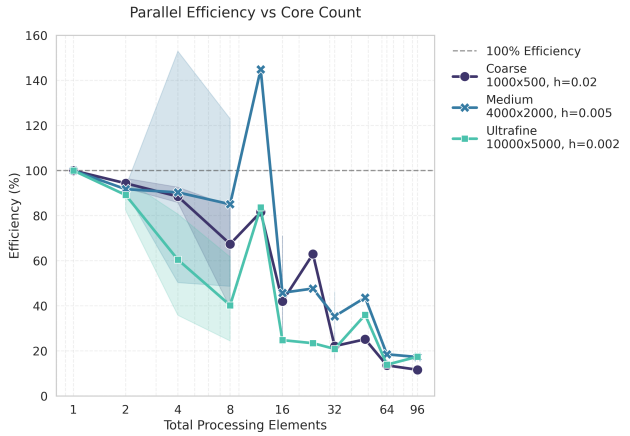
Despite these limitations, the solver exhibits stable and predicable scaling behaviour without performance collapse, indicating a well-balanced parallel implementation.



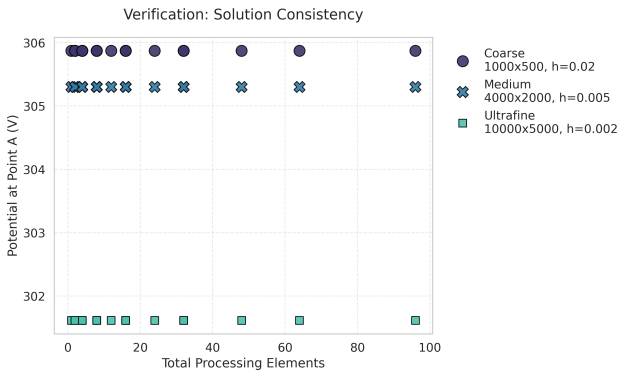
(a) Strong Scaling Speedup. Log-log plot showing deviation from ideal linear scaling (dashed line).



(b) Wall-Clock Runtime. Total solver execution time (s) vs. number of processing elements.



(c) Parallel Efficiency. Percentage of ideal speedup maintained as core count increases.



(d) Consistency Verification. Potential at monitoring Point A (V) remains constant across all configurations.

Figure 2: Scaling analysis of the hybrid solver across three grid resolutions. The Coarse grid (1000×500) saturates rapidly, while the Ultrafine grid (10000×5000) exhibits near-linear scaling up to 16 cores and significant gains up to 96 cores.

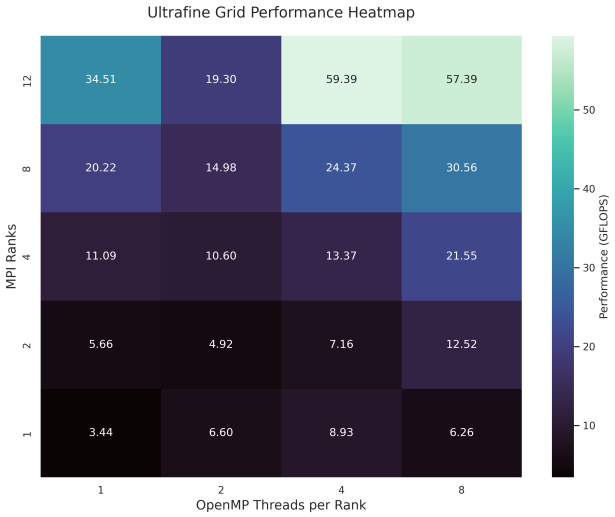


Figure 3: Performance heatmap for ultrafine ($h = 0.002$) grid for varying ranks and threads.

1.5.2 Runtime

Figure 2b presents the wall-clock solver runtime as a function of processing elements on a logarithmic scale. For all grid resolutions, runtime decreases monotonically with increasing core counts in the low-to-moderate parallel regime, closely following the ideal $1/N$ trend. This confirms that the dominant cost at small core counts is computational rather than communicational.

At higher processor counts, runtime reductions diminish, and, in some cases, become non-monotonic. This effect is most pronounced for the coarse grid, where runtime improvements largely saturate beyond 16 cores.

For the medium grid, useful runtime reductions persist to higher core counts, though fluctuations are visible. These fluctuations likely arise from load imbalance, communication latency, and MPI synchronisation effects, which become increasingly significant as the local domain size decreases. Notably, the medium grid exceeds 100% efficiency at 12 cores, confirming a cache-locality advantage where memory access speeds exceed those of the serial baseline. This corresponds to the 12 ranks, 1 thread configuration, suggesting that pure MPI scales well for the medium problem size.

The ultrafine grid consistently exhibits the largest absolute runtimes, but shows significant fluctuations. Between 2 and 8 processing elements the runtime increases, suggesting communication overhead outweighs the increased compute power. This drops dramatically back to linear-scaling at 12 processing elements (pure MPI). Beyond this, runtime increases but remains mostly stable, seeing meaningful reduction at high core counts.

1.5.3 Efficiency

Figure 2c shows the parallel efficiency. At low processor counts, all cases maintain efficiencies close to 100% demonstrating that the parallel overhead is minimal when each process owns a large portion of the domain. As the number of cores increases, efficiency decreases for all grid sizes, reflecting the growing impact of communication, synchronisation costs and reduced arithmetic intensity. The coarse grid experiences the most rapid efficiency degradation overall, falling below 50% by approximately 16 cores and reaching the lowest efficiencies at high core counts. This confirms that the coarse problem is not well-suited to large-scale parallel execution.

The medium grid maintains higher efficiency over a broader range of processor counts, though efficiency eventually drops as well. Interestingly, we again see a spike at 12 processing elements (pure MPI) where super-efficiency ($\sim 150\%$) is achieved. All three problem sizes see efficiency increase with pure MPI (12 ranks), however, only the medium problem size achieves super-linear

scaling, suggesting that it maximises memory bandwidth. In contrast, the ultrafine grid size, peaks but is still sub-linear, suggesting that the problem size is too large to fit within the cache, forcing slow retrieval from main memory, lowering efficiency.

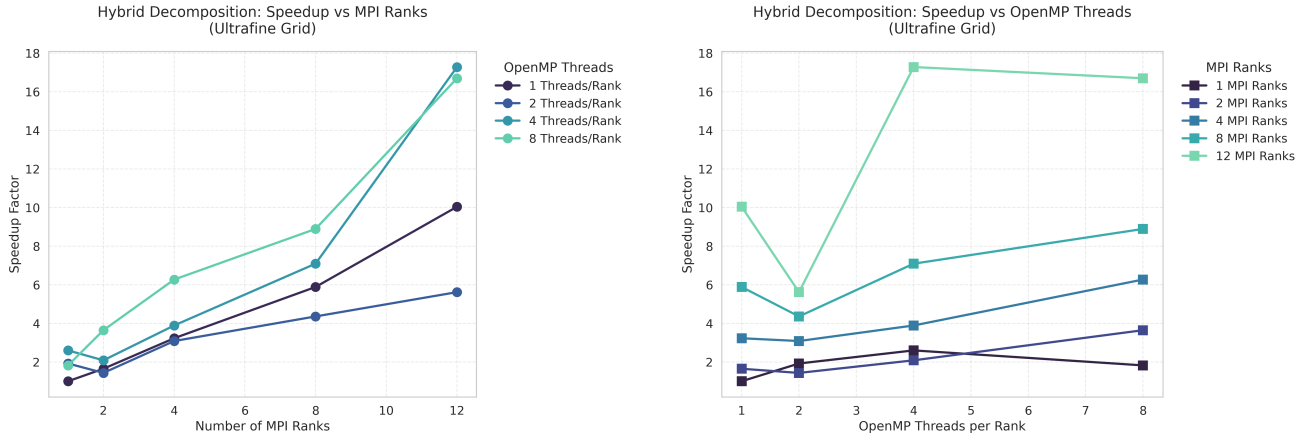
1.5.4 Performance Analysis

The performance heatmap (Fig 3) for the ultrafine grid shows that peak throughput (59.39 GFLOPS) is achieved at 12 ranks × 4 threads. On the Viking architecture (AMD EPYC 7643), this configuration is highly effective because each 4-thread rank fits entirely within a single 6-core Core Complex Die (CCD), maximising L3 cache.

At 12 ranks, increasing to 8 threads per rank causes a performance drop (from 59.39 to 57.39 GFLOPS) because 8 threads exceed the 6-core CCD boundary. This identifies a NUMA cliff, where threads are forced to communicate via the slower infinity fabric across CCDs.

Pure MPI scaling performs exceedingly well, with 12 ranks, 1 thread (34.51 GFLOPS) providing a (10×) speedup over 1 rank, 1 thread (3.44 GFLOPS). This even beats heavily hybridised configurations, such as 8 ranks, 8 threads at 30.56 GFLOPS. Hybrid configurations only outperform pure MPI for the configurations of 12 ranks × 4 threads and 12 ranks × 8 threads, suggesting that the increased communication and synchronisation costs in hybrid configurations hurt performance until significant core counts are reached, where compute power offsets such costs.

1.5.5 MPI vs. OpenMP



(a) Speedup vs MPI Ranks. Shows scaling behaviour when increasing domain decomposition while keeping thread count fixed.

(b) Speedup vs OpenMP Threads. Demonstrates the effect of increasing shared-memory parallelism for a fixed number of MPI ranks.

Figure 4: Hybrid decomposition analysis for the Ultrafine grid (10000 × 5000). Figure (a) illustrates that increasing MPI ranks generally yields consistent speedup. Figure (b) reveals that increasing threads per rank is effective only up to a point, often showing diminishing returns or slowdowns (e.g. at 8 threads) due to crossing the CCD boundary.

Considering a comparison between MPI and OpenMP performance, the heatmap (Fig 3) confirms that increasing MPI ranks (vertical movement) generally provides more robust performance gains that increasing threads (horizontal movement). At 8 total cores, 8 ranks × 1 thread (20.22 GFLOPS) vastly outperforms 1 rank × 8 threads (6.25 GFLOPS), proving that the solver benefits more from the distributed memory controllers of the multi-rank approach.

This finding is further supported by the MPI and OpenMP scaling charts (Figures 4a and 4b respectively). Increasing the number of MPI ranks generally has the greatest effect on performance, showing significant gains at 12 ranks.

OpenMP performance improvements are significantly more modest compared to MPI, performance rises from 3.44 GFLOPS at 1 thread to 8.93 GFLOPS at 4 threads (a sub-linear 2.4× increase). A significant performance drop is observed when moving from 4 to 8 threads, where GFLOPS decrease from 8.93 to 6.26, this suggests a NUMA cliff, where the increase to 8 threads must span across 2 CCDs, increasing communication overhead.

Figure 2d shows the stability of the potential result (at point A) for the three problem sizes as the number of total processing elements is increased. For all three problem sizes the potential remains stable. We see that as the problem size is increased, the potential reduces, suggesting the finer grid size enables a higher level of accuracy, as expected.

1.5.6 Conclusion

In conclusion, the data demonstrates that while increasing MPI ranks is the most reliable way to scale, hybrid configurations can outperform pure MPI but only for large problem sizes and high core counts. A balanced hardware approach that respects hardware boundaries (avoiding CCD crossing) is necessary to reach peak performance.

1.5.7 Additional Exploration: Ultimate Performance Run

The scaling analysis presented in the previous section identified two primary performance bottlenecks, memory bandwidth saturation and the NUMA cliff associated with crossing CCD boundaries. To provide a final proof of these conclusions, a targeted ultimate performance run was conducted using a configuration specially designed to align with the hardware topology of the AMD EPYC 7643 node.

As proof of concept, the solver was executed with 16 MPI ranks and 6 OpenMP threads per rank, utilising the full 96-core capacity of a dual-socket node. This specific configuration was chosen as each socket contains 8 CCDs, and each CCD contains 6 physical cores sharing a local L3 cache. By providing 6 threads per rank, each rank is perfectly contained within a single CCD, ensuring that all threads within a rank share a local L3 cache, eliminating high-latency communication over the infinity fabric. Utilising 16 ranks populates all 16 CCDs across the node, providing the most efficient aggregate path to the system's memory controllers.

The program was executed for the ultrafine grid size ($h = 0.002$) with the same parameters as the scaling run (Table 4).

The results of this run provide the proof for the necessity of hardware-aware configurations. The solver achieved a wall-clock time of 372.49 s, delivering a peak performance of ∼184.36 GFLOPS. This represents a 53.6× speedup over the serial baseline and a 3.21× improvement over the 12 × 8 configuration.

Table 6: Comparison of node-level parallel configurations for the ultrafine grid (10000×5000). The CCD-aligned configuration demonstrates the significant performance benefit of matching software threads to the hardware's physical chiplet boundaries.

Configuration	**Total Cores**	**Wall-Clock Time (s)**	**Performance (GFLOPS)**
Socket Aligned (12 × 8)	96	1196.57	57.39
CCD Aligned (16 × 6)	96	372.49	184.37

Despite both configurations utilising the same number of processing elements (96), the 300% performance difference is purely a function of topology alignment. This result serves as a definite conclusion to this study, for memory-bound solver such as SOR, topologically-aware placement, specifically matching the thread count to the physical hardware chiplets, is the single most significant factor in achieving high-concurrency performance.

1.5.8 Scripts and Code

Listing 5: Scaling script for Hybrid MPI+OpenMP program (coax_hybrid.f90) on Viking (run_scaling.sh)

```
#!/bin/bash
#SBATCH --job-name=scaling
#SBATCH --partition=nodes
#SBATCH --nodes=1
#SBATCH --ntasks=12
#SBATCH --cpus-per-task=8
#SBATCH --time=48:00:00
#SBATCH --output=scaling_%j.log

# Load the module
module purge
module load foss

# Compile code
mpif90 -O3 -fopenmp -o coax_hybrid coax_hybrid.f90

ranks=(1 2 4 8 12)
threads=(1 2 4 8)

for r in "${ranks[@]}"; do
        for t in "${threads[@]}"; do

                echo "MPI Ranks: $r"
                echo "OpenMP Threads: $t"

                export OMP_NUM_THREADS=$t
                export OMP_PLACES=cores
                export OMP_PROC_BIND=close

                time mpirun -np $r \
                --map-by socket:PE=$t \
                 ./coax_hybrid
        done
done
```

Listing 6: Single run script for Hybrid MPI+OpenMP program (coax_hybrid.f90) on Viking (run_single.sh).

```
#!/bin/bash
#SBATCH --job-name=r16_t6
#SBATCH --partition=nodes
#SBATCH --nodes=1
#SBATCH --ntasks=16
#SBATCH --cpus-per-task=6
#SBATCH --time=00:30:00
#SBATCH --output=scaling_r16_t6_%j.log

# Load the module
module purge
module load foss
```

```

# Compile code
mpif90 -O3 -fopenmp -o coax_hybrid coax_hybrid.f90

r=16
t=6

echo "MPI Ranks: $r"
echo "OpenMP Threads: $t"

export OMP_NUM_THREADS=$t
export OMP_PLACES=cores
export OMP_PROC_BIND=close

mpirun -np $r \
--map-by socket:PE=$t \
./coax_hybrid
```

Listing 7: Parallelised hybrid MPI + OpenMP code (coax_hybrid.f90)

```
module precision
    ! Double precision kind parameter
    implicit none
    integer, parameter :: dp = selected_real_kind(15, 300)
end module precision

module sim_params
    ! Simulation parameters
    use precision
    implicit none

    type :: params_t
        ! Geometry
        real(kind=dp) :: L_outer, L_inner
        real(kind=dp) :: R_outer, R_inner

        ! Boundary conditions
        real(kind=dp) :: phi_inner  ! Potential on inner conductor

        ! Solver settings
        real(kind=dp) :: h          ! Grid spacing
        real(kind=dp) :: omega      ! SOR relaxation factor
        real(kind=dp) :: tol        ! Convergence tolerance
        integer       :: max_iter   ! Maximum iterations
    end type params_t

    contains

        function default_params() result(p)
            ! Returns default simulation parameters
            type(params_t) :: p
            p%L_outer = 20.0_dp
            p%L_inner = 5.0_dp
            p%R_outer = 10.0_dp
            p%R_inner = 1.0_dp
```

```
            p%phi_inner = 1000.0_dp

            p%h        = 0.02_dp
            p%omega    = 1.9_dp
            p%tol      = 1.0e-4_dp
            p%max_iter = 200000
        end function default_params

end module sim_params

module potentials
    use precision
    use sim_params
    implicit none

    contains

        ! Analytical solution for coaxial capacitor at z=0
        pure function coax_analytic(r, p) result(val)
        ! Assumes r > R inner (Caller must ensure this)
            real(kind=dp), intent(in) :: r
            type(params_t), intent(in) :: p
            real(kind=dp) :: val

            ! Calculate potential
            val = (log(p%R_outer) - log(r)) / &
            (log(p%R_outer) - log(p%R_inner)) * p%phi_inner

        end function coax_analytic

        ! SOR update at r=0 (axis)
        pure function update_axis(phi_old, phi_right, phi_up, phi_down,
           omega) result(phi_new)
            ! Update potential at r=0 using SOR
            real(kind=dp), intent(in) :: phi_old    ! phi(i,0)
            real(kind=dp), intent(in) :: phi_right  ! phi(i,1)
            real(kind=dp), intent(in) :: phi_up     ! phi(i+1,0)
            real(kind=dp), intent(in) :: phi_down   ! phi(i-1,0)

            real(kind=dp), intent(in) :: omega
            real(kind=dp) :: phi_new, U

            ! Compute the update value U
            U = (2.0_dp/3.0_dp) * phi_right + &
            (1.0_dp/6.0_dp) * (phi_up + phi_down)

            ! SOR formula
            phi_new = phi_old + omega * (U - phi_old)

        end function update_axis

        ! SOR update at r>0
        pure function update_bulk(phi_old, p_left, p_right, p_up, p_down,
           inv_r_factor, omega) result(phi_new)
```

```
88
89              real(kind=dp), intent(in) :: phi_old        ! phi(i,0)
90              real(kind=dp), intent(in) :: p_left         ! phi(i,-1)
91              real(kind=dp), intent(in) :: p_right        ! phi(i,1)
92              real(kind=dp), intent(in) :: p_up           ! phi(i+1,0)
93              real(kind=dp), intent(in) :: p_down         ! phi(i-1,0)
94
95              real(kind=dp), intent(in) :: inv_r_factor   ! Avoid slow
                   division (1/8r)
96
97              real(kind=dp), intent(in) :: omega
98              real(kind=dp) :: phi_new, U
99
100             ! Compute the update value U
101             U = 0.25_dp * (p_left + p_right + p_up + p_down) + &
102             inv_r_factor * (p_right - p_left)
103
104             ! SOR formula
105             phi_new = phi_old + omega * (U - phi_old)
106
107         end function update_bulk
108
109 end module potentials
110
111 module grid_data
112     use precision
113     use sim_params
114     use potentials
115     use mpi
116     implicit none
117
118     ! Main potential array (allocates with halo cells)
119     real(kind=dp), allocatable :: phi(:,:)
120
121     ! Grid dimensions (global)
122     integer :: nz, nr                   ! Number of z and r points
123     integer :: nz_in, nr_in             ! Number of z and r points for
           inner conductor
124
125     ! MPI dimensions
126     integer :: nr_local                 ! Number of columns (r-indices) for
            this rank
127     integer :: nr_start                 ! Starting global r-index for this
           rank
128     integer :: rank, nprocs             ! MPI rank and number of processes
129     integer :: comm, ierr               ! MPI communicator and error code
130     integer :: left_rank, right_rank    ! Neighbour ranks
131
132     contains
133
134         subroutine init_grid(p, mpi_comm)
135             ! Initialise grid dimensions and MPI params
136             type(params_t), intent(in) :: p
137             integer, intent(in) :: mpi_comm
```

```
138             integer :: i, k_local, k_global, remainder
139             real(kind=dp) :: r_val
140
141             comm = mpi_comm
142             call MPI_Comm_rank(comm, rank, ierr)
143             call MPI_Comm_size(comm, nprocs, ierr)
144
145             ! Global grid size
146             nz = nint(p%L_outer / p%h)
147             nr = nint(p%R_outer / p%h)
148             nz_in = nint(p%L_inner / p%h)
149             nr_in = nint(p%R_inner / p%h)
150
151             ! Domain decomposition (split R)
152             nr_local = nr / nprocs
153             remainder = mod(nr, nprocs)
154
155             if (rank < remainder) nr_local = nr_local + 1
156
157             if (rank < remainder) then
158                 nr_start = rank * nr_local
159             else
160                 nr_start = rank * nr_local + remainder
161             end if
162
163             left_rank = rank - 1
164             right_rank = rank + 1
165             if (rank == 0) left_rank = MPI_PROC_NULL
166             if (rank == nprocs - 1) right_rank = MPI_PROC_NULL
167
168             ! Allocate (local + 2 halo columns)
169             allocate(phi(0:nz, 0:nr_local+1))
170             phi = 0.0_dp
171
172             ! Apply boundary conditions
173             do k_local = 1, nr_local
174                 k_global = nr_start + k_local
175                 r_val = real(k_global, dp) * p%h
176
177                 ! Inner conductor (r <= R_inner)
178                 if (k_global <= nr_in) then
179                     do i = 0, nz_in
180                         phi(i, k_local) = p%phi_inner
181                     end do
182                 end if
183
184                 ! Inlet BC (z=0) between inner/outer radius
185                 if (k_global > nr_in) then
186                     phi(0, k_local) = coax_analytic(r_val, p)
187                 end if
188             end do
189
190             ! Axis BC (r=0) on rank 0 only
191             if (rank ==0) then
```

```
192             phi(0:nz_in, 0) = p%phi_inner
193         end if
194
195     end subroutine init_grid
196
197     subroutine probe_point(r_target, z_target, name, p)
198         real(kind=dp), intent(in) :: r_target, z_target
199         character(len=*), intent(in) :: name
200         type(params_t), intent(in) :: p
201         integer :: iz, ir, local_r
202         real(kind=dp) :: local_phi, global_phi
203
204         iz = nint(z_target / p%h)   ! z-index
205         ir = nint(r_target / p%h)   ! r-index
206
207         ! Default invalid value
208         local_phi = -1.0_dp
209
210         ! Check axis (Only Rank 0 has this)
211         if (ir == 0) then
212             if (rank == 0) local_phi = phi(iz, 0)
213
214         ! Check bulk (Find the rank who owns this column)
215         else if (ir > nr_start .and. ir <= nr_start + nr_local) then
216             local_r = ir - nr_start
217             local_phi = phi(iz, local_r)
218         end if
219
220         ! REDUCE: Send the highest value found to rank 0
221         ! Ensures order of results
222         call MPI_Reduce(local_phi, global_phi, 1, MPI_DOUBLE_PRECISION,
                MPI_MAX, 0, comm, ierr)
223
224         ! Only rank 0 prints
225         if (rank == 0) then
226             print '(3A, F10.4, A)', 'Point ', name, ':', global_phi, '
                    V'
227         end if
228     end subroutine probe_point
229
230     subroutine save_parallel_dat(p)
231         type(params_t), intent(in) :: p
232         character(len=64) :: filename
233         integer :: unit_num, i, k, k_global
234
235         ! Unique filename for each rank
236         write(filename, '(A,I0,A)') 'phi_rank_', rank, '.dat'
237
238         open(newunit=unit_num, file=filename, status='replace', action=
                'write')
239
240         ! Axis (Rank 0 only)
241         if (rank == 0) then
242             do i = 0, nz
```

```
                    ! Z, R, Phi (R=0 here)
                    write(unit_num, *) real(i,dp)*p%h, 0.0_dp, phi(i,0)
                end do
            end if

            ! Bulk
            do k = 1, nr_local
                k_global = nr_start + k
                do i = 0, nz
                    write(unit_num, *) real(i,dp)*p%h, real(k_global,dp)*p%
                       h, phi(i,k)
                end do
            end do

            close(unit_num)

            if (rank == 0) print *, 'Parallel .dat write complete.'
        end subroutine save_parallel_dat

        subroutine cleanup()
            ! Deallocate grid
            if (allocated(phi)) deallocate(phi)
        end subroutine cleanup

end module grid_data

module sor_solver
    use precision
    use sim_params
    use grid_data
    use potentials
    use mpi
    implicit none

contains

    subroutine solve(p)
        type(params_t), intent(in) :: p

        integer :: iter, phase, i, k, k_global
        real(kind=dp) :: phi_old, phi_new
        real(kind=dp) :: local_diff, global_diff, diff
        real(kind=dp) :: inv_r_factor

        integer :: requests(4)
        integer :: statuses(MPI_STATUS_SIZE, 4)
        integer :: ierr

        if (rank == 0) then
            print *, 'Starting SOR...'
            print *, repeat('-',30)
        end if

        do iter = 1, p%max_iter
```

```
296             local_diff = 0.0_dp
297
298             do phase = 0, 1
299
300                 ! Start non-blocking communications
301                 ! Post recvs
302                 call MPI_Irecv(phi(0, 0), nz+1, MPI_DOUBLE_PRECISION,
                       left_rank, 1, &
303                                comm, requests(1), ierr)
304                 call MPI_Irecv(phi(0, nr_local+1), nz+1,
                       MPI_DOUBLE_PRECISION, right_rank, 2, &
305                                comm, requests(2), ierr)
306
307                 ! Post sends
308                 call MPI_Isend(phi(0, 1), nz+1, MPI_DOUBLE_PRECISION,
                       left_rank, 2, &
309                                comm, requests(3), ierr)
310                 call MPI_Isend(phi(0, nr_local), nz+1, MPI_DOUBLE_PRECISION
                       , right_rank, 1, &
311                                comm, requests(4), ierr)
312
313
314                 ! Axis update [r=0] (rank 0 only - strictly local)
315                 if (rank == 0) then
316                     k = 0
317                     !$OMP PARALLEL DO &
318                     !$OMP PRIVATE(i, phi_old, phi_new, diff) &
319                     !$OMP REDUCTION(max: local_diff)
320
321                     ! Update along axis
322                     do i = nz_in + 1, nz - 1
323                         if (mod(i, 2) /= phase) cycle
324
325                         ! SOR update at axis
326                         phi_new = update_axis(phi(i,0), phi(i,1), phi(i
                               +1,0), phi(i-1,0), p%omega)
327
328                         ! Store new value and track max difference
329                         phi_old = phi(i,0)
330                         phi(i, 0) = phi_new
331                         diff = abs(phi_new - phi_old)
332                         if (diff > local_diff) local_diff = diff
333                     end do
334                     !$OMP END PARALLEL DO
335                 end if
336
337                 ! B. Bulk Update
338                 if (nr_local >= 2) then
339                     !$OMP PARALLEL DO &
340                     !$OMP PRIVATE(k, k_global, inv_r_factor, i, phi_old,
                           phi_new, diff) &
341                     !$OMP REDUCTION(max: local_diff) &
342                     !$OMP SCHEDULE(DYNAMIC)
343
```

```
344                     ! Update internal columns
345                     do k = 2, nr_local - 1
346                         k_global = nr_start + k
347 
348                         ! Optimisation: Compute division once per column,
                               pass as multiplier
349                         inv_r_factor = 1.0_dp / (8.0_dp * real(k_global, dp
                               ))
350 
351                         ! Update along z
352                         do i = 1, nz - 1
353                             ! Skip inner conductor region
354                             if (i <= nz_in .and. k_global <= nr_in) cycle
355 
356                             ! Red-Black ordering
357                             if (mod(i + k_global, 2) /= phase) cycle
358 
359                             ! SOR update at bulk
360                             phi_new = update_bulk(phi(i,k), phi(i,k-1), phi
                                   (i,k+1), &
361                                                   phi(i+1,k), phi(i-1,k), &
362                                                   inv_r_factor, p%omega)
363 
364                             ! Store new value and track max difference
365                             phi_old = phi(i,k)
366                             phi(i, k) = phi_new
367                             diff = abs(phi_new - phi_old)
368                             if (diff > local_diff) local_diff = diff
369                         end do
370                     end do
371                     !$OMP END PARALLEL DO
372                 end if
373 
374                 ! Wait for comms to complete
375                 call MPI_Waitall(4, requests, statuses, ierr)
376 
377 
378                 ! C. Halo Update
379                 !$OMP PARALLEL DO &
380                 !$OMP PRIVATE(k, k_global, inv_r_factor, i, phi_old,
                       phi_new, diff) &
381                 !$OMP REDUCTION(max: local_diff)
382                 do k = 1, nr_local, max(1, nr_local - 1)
383                     if (k > 1 .and. k < nr_local) cycle
384 
385                     ! Determine global r-index
386                     k_global = nr_start + k
387                     if (k_global >= nr) cycle
388 
389                     ! Optimisation: Compute division once per column, pass
                           as multiplier
390                     inv_r_factor = 1.0_dp / (8.0_dp * real(k_global, dp))
391 
392                     ! Update along z
```

```
                    do i = 1, nz - 1
                        if (i <= nz_in .and. k_global <= nr_in) cycle
                        if (mod(i + k_global, 2) /= phase) cycle

                        ! SOR update at bulk
                        phi_new = update_bulk(phi(i,k), phi(i,k-1), phi(i,k
                           +1), &
                                              phi(i+1,k), phi(i-1,k), &
                                              inv_r_factor, p%omega)

                        ! Store new value and track max difference
                        phi_old = phi(i,k)
                        phi(i, k) = phi_new
                        diff = abs(phi_new - phi_old)
                        if (diff > local_diff) local_diff = diff
                    end do
                end do
                !$OMP END PARALLEL DO

            end do

            ! Global convergence
            call MPI_Allreduce(local_diff, global_diff, 1,
               MPI_DOUBLE_PRECISION, &
                               MPI_MAX, comm, ierr)
            if (rank == 0) then
                if (iter == 1 .or. mod(iter, 500) == 0) then
                    ! Format: 'CONV' tag, Iteration, Error
                    print '(A, I8, E14.6)', 'CONV ', iter, global_diff
                end if
            end if

            ! Exit if converged
            if (global_diff < p%tol) exit
        end do

        ! Final output
        if (rank == 0) then
            print *, repeat('-',30)
            print *, 'Converged in', iter, 'iterations. Final error:',
               global_diff
        end if
    end subroutine solve

end module sor_solver

program main
    use precision
    use sim_params
    use grid_data
    use potentials
    use sor_solver
    use mpi
    use omp_lib
```

```
    implicit none

    type(params_t) :: params
    real(kind=dp) :: t_start, t_end

    call MPI_Init(ierr)

    ! Set up params
    params = default_params()

    ! User overrides (for example)
    params%h        = 0.002_dp
    params%omega    = 1.98_dp
    params%tol      = 1.0e-4_dp
    params%max_iter = 1000000

    ! Initialise the grid
    call init_grid(params, MPI_COMM_WORLD)

    ! Print configuration (Only Rank 0 prints)
    if (rank == 0) then
        print *, repeat('=', 30)
        print *, 'SIMULATION PARAMETERS'
        print *, repeat('=', 30)
        print *, 'Geometry:'
        print *, '  L_outer:  ', params%L_outer
        print *, '  L_inner:  ', params%L_inner
        print *, '  R_outer:  ', params%R_outer
        print *, '  R_inner:  ', params%R_inner
        print *, '  phi_inner:', params%phi_inner, ' V'
        print *
        print *, 'Solver Settings:'
        print *, '  h:        ', params%h
        print *, '  omega:    ', params%omega
        print *, '  tol:      ', params%tol
        print *, '  max_iter: ', params%max_iter

        print *, repeat('-', 30)
        print *, 'Parallel Config:'
        print *, '  MPI Ranks:     ', nprocs
        print *, '  OpenMP Threads:', omp_get_max_threads()
        print *, '  Global Grid:   ', nz, 'x', nr
        print *, '  Total Points:  ', nz * nr
        print *, repeat('=', 30)
    endif

    ! Run solver
    call MPI_Barrier(MPI_COMM_WORLD, ierr)
    t_start = MPI_Wtime()
    call solve(params)
    t_end = MPI_Wtime()

    if (rank == 0) print *, 'Solver Time:', t_end - t_start, 's'

```

```
498     ! Check results
499     if (rank == 0) print *, '--- Final Results ---'
500     call probe_point(0.0_dp,  7.5_dp,  'A', params)
501     call probe_point(6.0_dp,  5.0_dp,  'B', params)
502     call probe_point(5.0_dp, 12.5_dp, 'C', params)
503 
504     ! Save the field for plotting
505     call save_parallel_dat(params)
506 
507     ! Cleanup and close MPI
508     call cleanup()
509     call MPI_Finalize(ierr)
510 
511 end program main
```